

HERBACAM USER GUIDE

Backend setup · Frontend setup · Application walkthrough

Alchornea cordifolia
(Euphorbiaceae)

Vernonia amygdalina
(Asteraceae)
Bitter leaf used for
diabetes and
infections
(Cameroon)

Piper guianense
(Piperaceae)
Used for various
problems in
the stomach and
(Cameroon)

The app is branded Ancestor in its interface; HerbaCam is the project name.

Local development and user workflows · Revision 6 October 2026

Contents

- 1. What HerbaCam does 2
- 2. What you need before starting 2
- 3. Start the backend 2
- 4. Start the frontend 5
- 5. First launch and demo sign-in 5
- 6. Navigation, sign-in, and shared controls 6
- 7. Explore the public application 6
- 8. Identify a plant with AI 7
- 9. Regular-user dashboard 7
- 10. Patient consultations 7
- 11. Traditional medicine practitioner workflow 8
- 12. Expert / reviewer workflow 9
- 13. Administrator workflow 9
- 14. Troubleshooting 11
- 15. Quick reference 11
- 16. Security, data, and responsible use 12

Section names in this contents list link to their place in the PDF.

1. What HerbaCam does

HerbaCam is a web application for exploring and documenting Cameroonian medicinal plant knowledge. It provides a public plant library, symptom search, an interactive regional map, educational articles, authenticated AI plant identification, a grounded question-and-answer assistant, and role-specific workspaces for community contributions, expert review, administration, and consultations.

The application currently displays the name Ancestor in its interface. The repository and project are named HerbaCam; the two names refer to this same application.

How the pieces connect

```
Browser at http://localhost:5173
├─ React / Vite frontend
├─ /api and /media requests are proxied by Vite
└─
Django REST API at http://localhost:8000
├─ MySQL / MariaDB (project database); SQLite is a local fallback
└─ OpenRouter for AI features (optional; requires a key)
```

The frontend does not talk directly to the database or AI provider. Django handles API requests, user permissions, database access, and AI-provider requests.

Important — educational use only. HerbaCam is not a diagnostic or treatment service. Traditional-use records are cultural and educational documentation; AI identification can be wrong. Do not use the application, its suggested plants, or any displayed dosage as a substitute for advice from a qualified health professional.

2. What you need before starting

Component	Requirement	Needed for
Python	Python 3.10–3.12 recommended	Django backend
Node.js and npm	Node 20.19+ or Node 22.12+	Vite / React frontend
Database	MySQL 5.7+ or MariaDB 10.4+ (project default); SQLite is an optional local fallback	Persistent application data
Browser	Current Chrome, Edge, Firefox, or Safari	Using the website
OpenRouter API key	Optional	AI plant identification and AI assistant replies
HTTPS or localhost	Needed for browser camera/microphone and location permission	Video/voice calls and location-aware features

Run `python --version` and `node --version` to check the installed versions. Install Node.js with npm included. Keep the repository root open in your terminal; the `backend` and `frontend` directories are siblings.

3. Start the backend

The backend is a Django REST API. Start it first; the frontend relies on it for data and sign-in.

Option A — MySQL or MariaDB (recommended for this project)

HerbaCam is configured to use MySQL/MariaDB by default when you use `backend/.env.example`. Start your local MySQL or MariaDB service first (for example, through XAMPP/WAMP or your system service). The supported versions are MySQL 5.7+ or MariaDB 10.4+.

1. Choose the database schema name. The example uses herbacam. If it does not already exist, create it in phpMyAdmin or a MySQL client:

```
CREATE DATABASE IF NOT EXISTS herbacam CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

If your existing database has another name, use that name in `DB_NAME` below instead. Ensure the MySQL user has permission to create and alter tables in that schema.

2. From the repository root, enter `backend/` and create your private environment file:

```
cd backend
cp .env.example .env
```

On Windows PowerShell, use `Copy-Item .env.example .env` in place of `cp`.

3. Edit `backend/.env`. The example already sets `DB_ENGINE=mysql`; set the actual database credentials and a private development `SECRET_KEY`:

```
DB_ENGINE=mysql
DB_NAME=herbacam
DB_USER=root
DB_PASSWORD=your-mysql-password
DB_HOST=127.0.0.1
DB_PORT=3306
```

Replace `DB_NAME`, `DB_USER`, and `DB_PASSWORD` with the values for your installation. If your local MySQL account has no password, set `DB_PASSWORD=`. Do not commit or share `.env` or real credentials. For production, use a dedicated least-privilege database user rather than a root account.

4. Create and activate the Python virtual environment, then install the backend dependencies.

Windows PowerShell:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If PowerShell blocks activation, use `.venv\Scripts\activate.bat` from Command Prompt.

macOS / Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

Then, on any platform:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

5. Check the Django configuration and apply database migrations:

```
python manage.py check
python manage.py migrate
```

6. Only if this is a new, disposable development database, add the sample dataset and its plant images:

```
python manage.py seed_data --clear
```

Skip this command if the MySQL database contains any records you need to keep. The seeder's `--clear` option deletes application data, including users, before rebuilding the demo dataset.

7. Start the backend:

```
python manage.py runserver 0.0.0.0:8000
```

Keep this terminal open. Django should report that it is serving on port 8000. Open `http://localhost:8000/api/plants/` in a browser to check the API.

The backend prefers `mysqlclient` and has a PyMySQL fallback for common development environments. If MySQL reports “access denied,” recheck the user/password and grants. If it reports “unknown database,” recheck `DB_NAME` and create that schema. Confirm the database service is running on the `DB_HOST/DB_PORT` configured in `.env`.

Option B — SQLite fallback for isolated local testing

Use SQLite only when you want a separate local database without running MySQL. Django falls back to `backend/db.sqlite3` when no database configuration is present. Do not use this option if you intend the application to connect to your MySQL database.

If you copied `.env.example`, remove or comment out all `DB_*` lines and any `DATABASE_URL` in `backend/.env`; otherwise Django will select MySQL. Keep the virtual environment and installed requirements from Option A, then run from `backend/`:

```
python manage.py migrate
# Optional only for an empty, disposable local database:
python manage.py seed_data --clear
python manage.py runserver 0.0.0.0:8000
```

Sample data and plant images

`python manage.py seed_data --clear` fills an empty development database with demo users, plants, symptoms, articles, example contributions, and other records used to exercise the screens. It also copies plant images into `backend/media/`.

Destructive command. `--clear` deletes existing application records (including accounts) before rebuilding the demo dataset. Use it only with a disposable local database. Do not run it on a database containing data you want to keep.

If you skip the seeder, the application can run but many lists will be empty. If you want demo data but need to preserve your existing MySQL records, create a separate schema such as `herbacam_demo`, set `DB_NAME=herbacam_demo`, run migrations, and seed that disposable schema instead. If seeded plant pictures return 404, run the seeder against the local demo database and confirm that `backend/media/` was created.

Optional — enable AI features

Plant identification and the assistant send requests through OpenRouter. Browsing the catalogue and using most non-AI features do not need an OpenRouter key. To enable the AI features, create or update `backend/.env` with values like these:

```
SECRET_KEY=replace-with-a-private-development-secret
DEBUG=True
OPENROUTER_API_KEY=your-openrouter-api-key
OPENROUTER_MODEL=google/gemini-3.8-flash
# Set this only if the selected model does not accept image input:
# OPENROUTER_VISION_MODEL=your-vision-capable-model-id
```

For MySQL/MariaDB, keep the `DB_*` configuration from Option A and add the AI values. For SQLite, keep database variables such as `DB_ENGINE`, `DB_HOST`, and `DATABASE_URL` out of this file. Restart the backend

after changing `.env`. Never put an OpenRouter secret in `frontend/.env` or a `VITE_` variable; frontend environment values are exposed to the browser.

The identification path requires a model that accepts images. If your configured model refuses image input, set `OPENROUTER_VISION_MODEL` to an image-capable model ID. The optional backend check is:

```
python manage.py check_identification
```

4. Start the frontend

Leave the backend terminal running. Open a second terminal in the repository root:

```
cd frontend
npm install
npm run dev
```

Vite prints the local development URL. Open `http://localhost:5173` in your browser. If port 5173 is already in use, use the URL printed by Vite.

The frontend is configured with `VITE_API_URL=/api`. Vite proxies `/api` and `/media` to `http://localhost:8000`, so the backend must be running on port 8000. For a normal local setup, do not replace `/api` with a browser-side `localhost` URL.

Keep both terminal windows open while using the application. To stop either server, focus its terminal and press `Ctrl+C`. These are development servers, not production hosting. The current development settings are permissive; do not expose them as a public production deployment without a separate security configuration.

Optional frontend checks, from `frontend/`:

```
npm run lint
npm run build
```

5. First launch and demo sign-in

After `migrate and seed_data --clear`, open `http://localhost:5173`, choose Log in, and use one of these local-only demo accounts:

Role	Username	Password
Administrator	admin	admin123!
Expert / reviewer	drnkeng	expert123!
Traditional medicine practitioner	mbaforc	pract123!
Regular user	demo_user	user1234!

The login page also provides quick demo-login buttons. These credentials exist only after the demo data is seeded. They are public development credentials: never use them on a live deployment. The seeder creates additional sample accounts; the passwords follow the same role-based patterns shown in the project README.

If you need a non-demo account, choose Get Started and provide your name, username, email, password, and account type. The public registration form offers Regular User, Traditional Medicine Practitioner, and Specialized Expert. Administrator accounts cannot be self-registered. A new expert listing begins unverified; an administrator must verify it before it is shown as verified in the specialist directory.

6. Navigation, sign-in, and shared controls

Public navigation

The top navigation provides Plants, Symptoms, Identify, Map, Articles, and About, along with Log in and Get Started when signed out. Visitors can browse public records without an account. All identification, saving favorites, and all dashboard workspaces require sign-in.

Dashboard navigation

After sign-in, the application routes you to a dashboard based on your account role. The left sidebar is different for users, practitioners, experts, and administrators. On a narrow screen, use the menu button to open it. The top bar contains:

- Bell: recent notifications; mark one or all as read.
- Profile menu: open your profile, return to the public site, or log out.
- Feedback control: signed-in users can send a bug report, content correction, data issue, suggestion, or other feedback. Administrators review it in the feedback workspace.

Pages are protected by role. If an account does not have permission for a workspace, the application sends it back to an accessible dashboard.

7. Explore the public application

Browse the plant library

1. Choose Plants in the navigation.
2. Enter a plant name in the search field, or use the filters for region, habitat, family, plant part, and evidence level.
3. Select a plant card to open its detail page.
4. Review the common and scientific names, local names, regions, documented traditional uses, plant part and preparation information, evidence records, and safety information that are available for that record.
5. If signed in, use the heart button to add or remove the plant from Favorites.

Some entries may not have an evidence or safety record yet. A missing record does not mean a plant is proven safe or effective. Dosage, frequency, duration, and administration text—when present—are part of a documented record, not instructions or a recommendation for the reader.

Search by symptom

1. Choose Symptoms.
2. Type a keyword such as a symptom name and select Search, or select one of the popular symptom links.
3. Open a result to view the associated plant record and its documented context.

Search results mean that a traditional use is documented in the database. They do not confirm a diagnosis or establish that the plant treats a condition.

Explore the geographic map

1. Choose Map.
2. Select Regions to explore regional summaries or Plants to see plant markers.
3. Select a region or marker to inspect the linked plant/knowledge records.
4. Use my location asks the browser for permission and centers the map on the nearest documented Cameroonian region. You can decline permission and explore the map manually.

Location access depends on browser permission and a secure context (<https://> or `localhost`). Map tiles may also require an internet connection.

Read articles and learn about the project

Choose Articles to browse published educational articles and open one to read it. Draft articles are not public. Choose About for the project overview.

8. Identify a plant with AI

Plant identification requires a signed-in account, a running backend, internet access, and a configured OpenRouter key/model.

1. Sign in and choose Identify (or Identify Plant from your dashboard).
2. Select a photo or drag and drop it into the upload area. Accepted formats are JPEG, PNG, and WebP; the file must be no larger than 10 MB.
3. Check the preview, then press Identify Plant.
4. Read the returned candidate plant names, confidence estimates, traditional context, and safety/evidence information where available. Open the linked plant details for more context.
5. Treat the result as a possible match, not a confirmed species identification. If it is incorrect, use the report option so a reviewer can examine it.

Use a clear photo of the plant and do not upload an image you do not have permission to share. Results are probabilistic; an inconclusive result is possible. View previous results in History. From history you can reopen a result, report it, or delete it.

9. Regular-user dashboard

Regular users can use the dashboard sidebar to:

- Dashboard: see account activity and shortcuts.
- Identify Plant / Symptom Search / Browse Plants: use the public discovery tools while signed in.
- Favorites: review saved plant records or remove a favorite.
- History: revisit previous AI identifications; report a questionable result or delete an item.
- Articles: return to published reading material.
- Book Consultation / My Appointments: request a specialist appointment and manage existing appointments.
- Ask Ancestor: start a question-and-answer conversation using the application knowledge base.
- Notifications: review updates about submissions, reviews, appointments, or messages.
- Profile: update your name, email, phone, bio, profile photo, or password.

The assistant's answers are intended to be grounded in verified records and may link to cited plants. If nothing relevant is found, the assistant can flag that it is answering without a matching knowledge-base record. Dosage should come from documented records, not be inferred from an AI answer. Assistant conversations can be started, reopened, or deleted from the conversation list.

Profile photos must be JPEG, PNG, or WebP and under 5 MB. Use the profile page to change your password; you will need your current password.

10. Patient consultations

Consultations let a signed-in regular user request a time with a specialist. The specialist publishes open time windows; a booking is a request until the specialist confirms it.

Request a time

1. Open Book Consultation.
2. Optionally filter by specialisation, show verified specialists only, or select Near me to sort by distance. Location sharing is optional.
3. Review the specialist's profile, region, focus, verification badge, and available windows.
4. Select an open time, enter what you would like to discuss, then choose Request this window.
5. Open My Appointments to see whether the request is pending, confirmed, declined/cancelled, completed, or marked no-show. The specialist must confirm before a pending request is scheduled.

Manage or attend an appointment

- For a pending or confirmed booking, open the appointment controls to cancel or ask to move it to another open window from the same specialist. A patient-requested time change returns to pending for specialist confirmation; a move proposed by the specialist stays confirmed. To see a different specialist, cancel and submit a new booking.
- When a booking is confirmed, use its room link at the agreed time. The appointment has a private message thread for the two participants.
- Patients can start a video or voice-only call. Allow microphone/camera access when prompted. The browser needs HTTPS or localhost for media permissions.
- The media connection is peer-to-peer; the backend relays the call setup, not the audio/video itself. The default configuration uses a STUN server. Some networks, carrier NATs, or strict firewalls require a TURN relay; without one, use the room's chat if the media connection cannot be established.

For local two-account testing, sign in as the patient and specialist in separate browser profiles (or one regular and one private window). Use headphones to avoid audio feedback. Do not use a development server to conduct private real-world health consultations.

11. Traditional medicine practitioner workflow

A practitioner account can share community knowledge for expert review.

Submit a contribution

1. Open New Contribution.
2. Select an existing plant and symptom when possible. If a record is missing, fill in the proposed plant or symptom fields.
3. Add the local name and language, plant part, preparation method, and a clear description of how the preparation is traditionally used.
4. Add dosage details only when they are part of the source knowledge: amount, frequency, duration, and how it is administered. Also add cultural context, region, community, and supporting information where known.
5. Choose Submit for Review to send it to an expert, or Save Draft to continue later.

A use description is required. Clear source, location, preparation, and dosage context helps reviewers assess a contribution. Share only information you are authorized to document, and avoid presenting a cultural record as clinical advice.

Track and update contributions

Open My Contributions to see the status and reviewer comments. Common statuses include Draft, Submitted, Under Review, Revision Requested, Rejected, Approved, and Published. If a reviewer requests changes or rejects a submission for revision, open it, read the comments, edit the record, and resubmit it. A published contribution can become part of the public plant/symptom knowledge pages.

Use Profile to keep your personal and practitioner information current.

12. Expert / reviewer workflow

Experts review community submissions and curate records. The Expert Dashboard shows the pending review queue.

Review a contribution

1. Open Pending Reviews and choose a submission. Use the full review page to inspect the plant, use description, location, preparation, source, and dosage context.
2. Decide whether the information is sufficiently clear and supported for the knowledge-base workflow.
3. Choose Approve, Request Revision, or Reject. Add specific comments or a reason so the practitioner knows what to do next.
4. The contributor can be notified and may edit/resubmit when changes are needed. Approved records can be published into the public knowledge base.

Curate records and analytics

The expert sidebar provides:

- Knowledge: inspect knowledge records and submissions.
- Plant Library: add or edit plant information and manage its publication status.
- Articles: create or edit public reading-room articles; drafts remain private.
- Evidence / Safety: manage scientific evidence summaries and safety/precaution records associated with plants.
- Preservation / Analytics: review trends and preservation-risk indicators. The 0–100 score combines five component scores (0–20 each): contributor scarcity, knowledge recency, geographic concentration, documentation scarcity, and submission decline. Levels are LOW (0–33), MODERATE (34–66), and HIGH (67–100). Treat these as prioritization indicators, not scientific predictions.

Manage specialist availability

Open Consultation Desk to update your specialist listing (specialisation, region, focus, and whether you are accepting patients). Publish future availability windows with a date, start time, duration, and optional note. Past or overlapping windows are not accepted. Patients can book the open windows you publish.

Use the desk to confirm or decline pending requests, move a booking, open the room for a confirmed appointment, complete a consultation with a closing note, or record a no-show. Cancelling/completing/moving a booking returns the released time to the availability pool automatically.

An administrator controls specialist verification and suspension. Turning off patient availability removes the specialist and their windows from the patient booking page.

13. Administrator workflow

Administrators have a separate dashboard for platform oversight and content administration. The administrator sidebar includes:

Workspace	Typical task
Users	Review accounts, update roles, or deactivate/reactivate users; a reason is required when deactivating an active account
Plants / Knowledge / Articles	Curate plant records and traditional knowledge; create, edit, publish, or withdraw content

Workspace	Typical task
Symptoms	Maintain the symptom vocabulary used by searches and submissions
Practitioners	Review practitioner profiles
Evidence / Safety	Maintain scientific evidence and safety information
Geography	Maintain the regions, divisions, and communities used in records
Preservation / Analytics	Review platform metrics and preservation-risk assessments
Consultations	Oversee all bookings and platform-wide consultation statistics; verify or suspend specialist listings
Feedback	Triage user feedback, reply, and resolve items
Audit Logs	Review administrative and platform actions

Plant and article curation is shared with experts. Symptom vocabulary and platform-wide role, verification, audit, and consultation oversight are administrator-level responsibilities. Use care when changing existing vocabulary or records because they may be referenced by other submissions.

14. Troubleshooting

Symptom	Check / solution
python or manage.py cannot find Django	Confirm the terminal is in backend/, activate .venv, then install requirements.txt.
MySQL connection refused or unknown database	Start MySQL/MariaDB, confirm the database exists, and check the DB_* settings, user grants, and port in backend/.env. If DATABASE_URL is also set, verify that it points to the intended database because it takes precedence over the individual DB_* values.
Plant pages are empty	Run migrations and seed_data --clear on a disposable local database. Remember that this resets local records.
Plant images return 404	Run the local seeder to copy sample images into backend/media/; confirm the backend is running and Vite proxies /media.
Frontend says network/API error	Keep the backend running on port 8000. Check http://localhost:8000/api/plants/, then refresh the Vite page. Confirm you started Vite from frontend/.
npm is not recognized or Vite will not start	Install a supported Node.js version (20.19+ or 22.12+), open a new terminal, then run npm install in frontend/.
AI identification or assistant fails	Set a valid OPENROUTER_API_KEY in backend/.env, confirm the backend was restarted and has internet access, and check that the configured model is available. For image errors, configure OPENROUTER_VISION_MODEL.
Camera or microphone is blocked	Open the app at https:// or localhost, grant browser permissions, check that another app is not using the device, and retry. Try voice-only if the camera is unavailable.
A call stays on “connecting” or cannot reach the other person	Check both participants' device permissions and network/firewall. STUN-only configuration cannot relay media through every NAT; configure a TURN server in WEBRTC_ICE_SERVERS for restrictive/cross-network calls.
Location search does not work	Allow browser location permission on a secure origin, or clear the location filter and browse regions manually.
Demo login does not work	Confirm seed_data --clear completed successfully and you are using the matching username/password from this guide.

15. Quick reference

Start both servers

Terminal 1 — backend (from repository root):

```
cd backend
# Activate .venv first, then:
python manage.py runserver 0.0.0.0:8000
```

Terminal 2 — frontend (from repository root):

```
cd frontend
npm run dev
```

Then open <http://localhost:5173>.

Common backend commands

Run these from backend/ with the virtual environment active:

```
python manage.py check
python manage.py migrate
python manage.py seed_data --clear # destructive local demo reset
python manage.py check_identification # optional AI/model diagnostic
```

Useful local URLs

URL	Purpose
http://localhost:5173	HerbaCam / Ancestor frontend
http://localhost:8000/api/plants/	Example backend API endpoint
http://localhost:8000/admin/	Django's built-in admin site (separate from the application's role-based admin dashboard)

The regular application admin dashboard is reached by signing in with an **ADMIN** role account, for example the seeded **admin** account, then choosing Dashboard.

16. Security, data, and responsible use

- The included runserver and Vite commands are for local development. Do not expose them as a production service.
- Development defaults are permissive (`DEBUG=True`, open host/CORS settings). A public deployment needs its own hardened Django, database, HTTPS, static/media, CORS, secret, and frontend configuration.
- Keep backend API keys and database passwords in a private `backend/.env`; never commit them or put secrets in frontend variables.
- Do not use the seeded demo users on a public deployment. Do not run `seed_data --clear` on any database whose contents must be preserved.
- Local SQLite data lives in `backend/db.sqlite3`; uploaded and seeded media lives in `backend/media/`. Back up the database and media together if you need to keep local work.
- AI matches and summaries are not authoritative. Traditional uses, evidence levels, safety records, and preservation scores can be incomplete or based on sample data. Always consult qualified health professionals before making health decisions.